

ret-sync

This is from <https://github.com/Fusion86/ret-sync>
But I only include ghidra here.

ret-sync stands for Reverse-Engineering Tools SYNChronization. It is a set of plugins that help to synchronize a debugging session (WinDbg/GDB/LLDB/OllyDbg/OllyDbg2/x64dbg) with a disassembler (IDA/Ghidra/Binary Ninja). The underlying idea is simple: take the best from both worlds (static and dynamic analysis).

Debuggers and dynamic analysis provide us with:

- local view, with live dynamic context (registers, memory, *etc.*)
- built-in specialized features/API (ex: WinDbg's `!peb`, `!drvobj`, `!address`, *etc.*)

Disassemblers and static analysis provide us with:

- macro view over modules
- code analysis, signatures, types, *etc.*
- fancy graph view
- decompilation
- persistent storage of knowledge within IDBs/GPRs

Key features:

- synchronize graph and decompilation views with debugger's state
- no need to deal with ASLR, addresses are rebased on-the-fly
- pass data (comment, command output) from debugger to disassembler
- multiple IDBs/GPRs can be synced at the same time allowing to easily trace through multiple modules
- disassembler and debugger can be on different hosts / VMs

ret-sync is a fork of [qb-sync](#) that I developed and maintained during my stay at [Quarkslab](#).

Table of contents

- [Repository content](#)
- [General prerequisites](#)
- [Binary release](#)
- [ret-sync configuration](#)
- [Installation](#)

- Ghidra extension
- GNU gdb (GDB) installation
- Usage
 - **ret-sync** debugger commands
 - IDA usage
 - Ghidra usage
 - GNU gdb (GDB) usage
 - Python library usage
- Extend
- TODO
- Known Bugs/Limitations
- License
- Greetz

Repository content

The debugger plugins:

- `ext_windbg/sync`: WinDbg extension source files, once built: `sync.dll`
- `ext_gdb/sync.py`: GDB plugin
- `ext_lldb/sync.py`: LLDB plugin
- `ext_olly1`: OllyDbg 1.10 plugin
- `ext_olly2`: OllyDbg v2 plugin
- `ext_x64dbg`: x64dbg plugin

The disassembler plugins:

- `ext_ida/SyncPlugin.py`
- `ext_ghidra/dist/ghidra_*_retsync.zip`: Ghidra plugin
- `ext_bn/retsync`: Binary Ninja plugin

PROF

And the library plugin:

- `ext_lib/sync.py`: standalone Python library

General prerequisites

IDA and GDB plugins require a valid Python setup. Python 2 (≥ 2.7) and Python 3 are supported.

Binary release

Pre-built binaries for WinDbg/OllyDbg/OllyDbg2/x64dbg debuggers are proposed through an **Azure DevOps** pipeline:  Azure Pipelines **succeeded**

Select the last build and check the artifacts under the **Related** section: **6 published**.

A pre-built plugin archive of the Ghidra plugin is provided in **ext_ghidra/dist**.

ret-sync configuration

ret-sync should work out of the box for most users with a typical setup: debugger and disassembler(s) on the same host, module names matching.

Still, in some scenarios a specific configuration may be used. For that, extensions and plugins check for an optional global configuration file named **.sync** in the user's home directory. It must be a valid **.INI** file.

Additionally, the IDA and Ghidra plugins also look for the configuration file in the IDB or project directory (**<project>.rep**) first to allow local, per-IDB/project, settings. If a local configuration file is present, the global configuration file is ignored.

Values declared in these configuration files override default values. Please note, that **no .sync** file is created by default.

Below we detail, three common scenarios where a configuration file is useful/needed:

- Remote debugging
- Modules names mismatch
- Missing PID

Remote debugging: debugger and disassembler are on different hosts

PROF

The **[INTERFACE]** section is used to customize network related settings. Let's suppose one wants to synchronize IDA with a debugger running inside a virtual machine (or simply another host), common remote kernel debugging scenario.

Simply create two **.sync** file:

- one on the machine where IDA is installed, in the IDB directory:
- For Ghidra, place at home directory, ex. **"/home/user"** or **"C:\Users\user"**.

```
[INTERFACE]
host=192.168.128.1
port=9234
```

It tells **ret-sync** IDA plugin to listen on the interface **192.168.128.1** with port **9234**. It goes without saying that this interface must be reachable from the remote host or virtual machine.

- one on the machine where the debugger is executed, in the user's home directory:

```
[INTERFACE]
host=192.168.128.1
port=9234
```

It tells **ret-sync** debugger plugin to connect to the **ret-sync** IDA plugin configured previously to listen in this interface.

NOTE: You must specify a real IP here, and not use **0.0.0.0**. This is because the variable is used by multiple sources both for binding and connecting, so using **0.0.0.0** will result in weird errors.

gdb with Qt Creator debugging frontend

The Qt Creator debugging frontend changes the way gdb command output is logged. Since this would interfere with the synchronization an option exists to use the raw gdb output for synchronization instead of a temporary file. In the **.sync** configuration file use

```
[GENERAL]
use_tmp_logging_file=false
```

if you wish to use the Qt debugging frontend for the target.

Embedded devices and missing **/proc/<pid>/maps**

PROF

In some scenarios, such as debugging embedded devices over serial or raw firmware in QEMU, gdb is not aware of the PID and cannot access **/proc/<pid>/maps**.

In these cases, The **[INIT]** section is used to pass a custom context to the plugin. It allows overriding some fields such as the PID and memory mappings.

.sync content extract:

```
[INIT]
context = { "pid": 200, "mappings": [ [0x400000, 0x7A81158, 0x7681158,
"asav941-200.qcow2|lina"] ] }
```

Each entry in the mappings is: **mem_base**, **mem_end**, **mem_size**, **mem_name**.

If doing this with ghidra:

- Make sure the context entry is a single line, or you will get parsing errors in the ghidra output window!
- Make sure the mem_name matches the name of the module (name next to folder icon in program tree window) exactly!

Bypassing automatic address rebasing

In some scenarios, such as debugging embedded devices or connecting to minimalist debug interfaces, it may be more convenient to bypass the automatic address rebasing feature implemented in the disassembler plugins.

The `use_raw_addr` option is currently supported only for Ghidra. In the .sync configuration file use:

```
[GENERAL]
use_raw_addr=true
```

Installation

Ghidra extension

Build the Ghidra extension

Either use the pre-built version from the `ext_ghidra/dist` folder or follow the instruction to build it. Each extension build only supports the version of Ghidra specified in the plugin's file name.

E.g. `ghidra_9.1_PUBLIC_20191104_retsync.zip` is for Ghidra 9.1 Public.

1. Install Ghidra
2. Install gradle

```
apt install gradle
```

3. Build extension for your Ghidra installation (replace `$GHIDRA_DIR` with your installation directory)

```
cd ext_ghidra
gradle -PGHIDRA_INSTALL_DIR=$GHIDRA_DIR
```

Install the Ghidra extension

1. From Ghidra projects manager: **File -> Install Extensions...**, click on the + sign and select the **ext_ghidra/dist/ghidra_*_retsync.zip** and click OK. This will effectively extract the **retsync** folder from the zip into **\$GHIDRA_DIR/Extensions/Ghidra/**
2. Restart Ghidra as requested
3. After reloading Ghidra, open a module in CodeBrowser. It should tell you a new extension plugin has been detected. Select "yes" to configure it. Then tick "RetSyncPlugin" and click OK. The console should show something like:

```
[*] retsync init
[>] programOpened: tm.sys
    imageBase: 0x1c0000000
```

4. From Ghidra CodeBrowser tool: use toolbar icons or shortcuts to enable (**Alt+s**)/disable (**Alt+Shift+s**)/restart (**Alt+r**) synchronization.

A status window is also available from **Windows -> RetSyncPlugin**. You generally want to drop it on the side to integrate it with the Ghidra environment windows.

GNU gdb (GDB) installation

1. Copy the **ext_gdb/sync.py** to the directory of your choice
2. Load the extension (see auto-load-scripts)

```
gdb> source sync.py
[sync] configuration file loaded 192.168.52.1:9100
[sync] commands added
```

PROF

Usage

ret-sync debugger commands

For command-line oriented debuggers (mainly Windbg and GDB) a set of commands is exposed by **ret-sync** to assist in the reverse-engineering task.

The commands below are generic (Windbg and GDB), please note that a ! prefix is needed on WinDbg (e.g.: **sync** in GDB, **!sync** in Windbg).

Debugger command	Description
synchelp	Display the list of available commands with short explanation
sync	Start synchronization

Debugger command	Description
<code>syncoff</code>	Stop synchronization
<code>cmt [-a address] <string></code>	Add a comment at current ip in disassembler
<code>rcmt [-a address]</code>	Reset comment at current ip in disassembler
<code>fcmt [-a address] <string></code>	Add a function comment for function in which current ip is located
<code>raddr <expression></code>	Add a comment with rebased address evaluated from expression
<code>rln <expression></code>	Get symbol from the disassembler for the given address
<code>lbl [-a address] <string></code>	Add a label name at current ip in disassembler
<code>cmd <string></code>	Execute a command in debugger and add its output as comment at current ip in disassembler
<code>bc <\\ \\on\\ off\\ set 0xBBGGRR></code>	Enable/disable path coloring in disassembler
<code>idblist</code>	Get list of all IDB clients connected to the dispatcher
<code>syncmodauto <on\\ off></code>	Enable/disable disassembler auto switch based on module name
<code>idbn <n></code>	Set active IDB to the nth client
<code>jmpto <expression></code>	
<code>jmprow <expression></code>	If an IDB is enabled then disassembler's view is synced with the resulting address.
<code>translate <base> <addr> <mod></code>	rebase an address with respect to its module's name and offset

PROF

GDB specific commands:

Debugger command	Description
<code>bbt</code>	Beautiful backtrace. Similar to bt in GDB but requests symbols from disassembler
<code>patch</code>	Patch bytes in disassembler based on live context
<code>bx</code>	Similar to GDB x but using a symbol. The symbol will be resolved by disassembler
<code>cc</code>	Continue to cursor in disassembler

Ghidra usage

Ghidra plugin's GUI

Once the RetSyncPlugin opened, you can add it to the CodeBrowser window by simple drag'n'drop:

If you want to view several modules, files need to be open in the same CodeBrowser viewer, simply drag'n'drop the additional ones in the CodeBrowser window to obtain the result as above.

Ghidra global shortcuts

ret-sync defines these global shortcuts in Ghidra:

- **Alt-S** - Enable syncing
- **Alt-Shift-S** - Disable syncing
- **Alt-R** - Restart syncing
- **Alt-Shift-R** - Reload configuration

Ghidra bindings over debugger commands

Bindings over debugger commands are also implemented. They are similar to the ones from IDA's extension (except the "Go" command).

- **F2** - Set breakpoint at cursor address
- **Ctrl-F2** - Set hardware breakpoint at cursor address
- **Alt-F3** - Set one-shot breakpoint at cursor address
- **Ctrl-F3** - Set one-shot hardware breakpoint at cursor address
- **Alt-F2** - Translate (rebase in debugger) current cursor address
- **F5** - Go
- **Alt-F5** - Run (GDB only)
- **F10** - Single step
- **F11** - Single trace

PROF

GNU gdb (GDB) usage

Sync with host:

```
gdb> sync
[sync] sync is now enabled with host 192.168.52.1
<not running>

gdb> r
Starting program: /bin/ls
[Thread debugging using libthread_db enabled]
Using host libthread_db library "/lib/libthread_db.so.1".
```


GDB plugin commands

Use commands, **without "!" prefix**

```
(gdb) cmd x/i $pc
[sync] command output: => 0x8049ca3:    push    edi

(gdb) synchelp
[sync] extension commands help:
> sync <host>
> syncoff
> cmt [-a address] <string>
> rcmt [-a address] <string>
> fcmt [-a address] <string>
> cmd <string>
> bc <on|off|>
> rln <address>
> bbt <symbol>
> patch <addr> <count> <size>
> bx /i <symbol>
> cc
> translate <base> <addr> <mod>
```

- **rln**: Get symbol from the IDB for the given address
- **bbt**: Beautiful backtrace. Similar to **bt** but requests symbols from disassembler

```
(gdb) bt
#0  0x0000000000a91a73 in ?? ()
#1  0x0000000000a6d994 in ?? ()
#2  0x0000000000a89125 in ?? ()
#3  0x0000000000a8a574 in ?? ()
#4  0x000000000044f83b in ?? ()
#5  0x0000000000000000 in ?? ()
(gdb) bbt
#0  0x0000000000a91a73 in IKE_GetAssembledPkt ()
#1  0x0000000000a6d994 in catcher ()
#2  0x0000000000a89125 in IKEProcessMsg ()
#3  0x0000000000a8a574 in IkeDaemon ()
#4  0x000000000044f83b in sub_44F7D0 ()
#5  0x0000000000000000 in ()
```

- **patch**: Patch bytes in disassembler based on live context
- **bx**: Beautiful display. Similar to **x** but using a symbol. The symbol will be resolved by disassembler.
- **cc**: Continue to cursor in disassembler. This is an alternative to using **F3** to set a one-shot breakpoint and **F5** to continue. This is useful if you prefer to do it from gdb.

```

(gdb) b* 0xA91A73
Breakpoint 1 at 0xa91a73
(gdb) c
Continuing.

Breakpoint 1, 0x0000000000a91a73 in ?? ()
(gdb) cc
[sync] current cursor: 0xa91a7f
[sync] reached successfully
(gdb)

```

Python library usage

One may want to use **ret-sync** core features (position syncing with a disassembler, symbol resolution) even though a full debugging environment is not available or with a custom tool. To that end, a minimalist Python library has been extracted.

The example below illustrates the usage of the Python library with a script that walks through the output of an event based logging/tracing tool.

```

from sync import *

HOST = '127.0.0.1'

MAPPINGS = [
    [0x555555400000, 0x555555402000, 0x2000, " /bin/tempfile"],
    [0x7ffff7dd3000, 0x7ffff7dfc000, 0x29000, " /lib/x86_64-linux-
gnu/ld-2.27.so"],
    [0x7ffff7ff7000, 0x7ffff7ffb000, 0x4000, " [vvar]"],
    [0x7ffff7ffb000, 0x7ffff7ffc000, 0x1000, " [vdso]"],
    [0x7ffff7ffde000, 0x7ffff7fff000, 0x21000, " [stack]"],
]

EVENTS = [
    [0x0000555555400e74, "malloc"],
    [0x0000555555400eb3, "open"],
    [0x0000555555400ee8, "exit"]
]

synctool = Sync(HOST, MAPPINGS)

for e in EVENTS:
    offset, name = e
    synctool.invoke(offset)
    print("    0x%08x - %s" % (offset, name))
    print("[>] press enter for next event")
    input()

```

Extend

While initially focused on dynamic analysis (debuggers), it is of-course possible to extend the plugins set and integrate with other tools.

- Integration with **REVEN** Timeless Analysis and Debugging Platform by [Tetrane](#):
 - <http://blog.tetrane.com/2015/02/reven-in-your-toolkit.html>
 - <https://twitter.com/tetrane/status/1374768014193799175>
- Integration with **EFI DXE Emulator** by Assaf Carlsbad ([@assaf_carlsbad](#)):
 - https://twitter.com/assaf_carlsbad/status/1242114356881641474
 - https://github.com/assafcarlsbad/efi_dxe_emulator

Other resource(s):

- "*Combining static and dynamic binary analysis - ret-sync*" by Jean-Christophe Delaunay
 - https://www.synacktiv.com/ressources/bieresecu1_ret-sync_en.pdf

TODO

- Sure.

Known Bugs/Limitations

- Tested with Python 2.7/3.7, IDA 7.7 (Windows, Linux and Mac OS X), Ghidra 10.1.1, Binary Ninja 3.0.3225-dev, GNU gdb (GDB) 8.1.0 (Debian), lldb 310.2.37.
- **THERE IS NO AUTHENTICATION/ENCRYPTION** whatsoever between the parties; you're on your own.
- Self modifying code is out of scope.

PROF

With GDB:

- it seems that stop event is not called when using 'return' command.
- multi-threading debugging have issues with signals.

With Ghidra:

- Shortcuts are not working as expected in decompiler widget.

Conflict(s):

- Logitech Updater software is known to use the same default port (9100). A solution is to use a global `.sync` configuration file to define a different port.

```
[INTERFACE]
host=127.0.0.1
```

License

ret-sync is free software: you can redistribute it and/or modify it under the terms of the GNU General Public License as published by the Free Software Foundation, either version 3 of the License, or (at your option) any later version.

This program is distributed in the hope that it will be useful, but WITHOUT ANY WARRANTY; without even the implied warranty of MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE. See the GNU General Public License for more details.

You should have received a copy of the GNU General Public License along with this program. If not, see <http://www.gnu.org/licenses/>.

The Binary Ninja plugin is released under the MIT licence.

Greetz

Hail to Bruce Dang, StalkR, @IvanleF0u, Damien Aumaître, Sébastien Renaud and Kévin Szkudlanski, @m00dy, @saidelike, Xavier Mehrenberger, ben64, Raphaël Rigo, Jiss for their kindness, help, feedbacks and thoughts. Ilfak Guilfanov, Igor Skochinsky and Arnaud Diederer for their help with IDA's internals and outstanding support. Thank you to Jordan Wiens and Vector 35. Finally, thank you also to all the contributors and everyone who reported issues/bugs.